Lab 2

Exercise 1. Using a button and 2 LEDs develop a system that can modify the LED status. The system should:

- State 1: Blink both LEDs alternating the sequence with 1 second intervals (when LED 1 is turned on LED 2 is turned off).
- State 2: Both LEDs have to blink simultaneously with 2 seconds intervals.
- State 3: In this state both LEDS have to be turned ON indefinitely.
- State 4: This state turns off both LEDs.

The system should return to state 1 after state 4.

Modify the code developed to work using functional programming.

Exercise 2. Using 4 LEDs and 2 buttons make a binary counter with the values from table 1. The counter should have the following features:

- When button 1 is pushed, the binary counter increases 1 value.
- When button 2 is pressed, the binary counter decreases 1 value.
- Each led represent the binary values.
- When the counter reaches the value 15 it should return to the first value
- When the counter is 0 it shouldn´t keep reducing the value (should stay in 0 after the button press)

Finally print in console the value in 3 representations (hex, binary, decimal).

Modify the code developed to work using OOP programming.

Exercise 3. We have a greenhouse that needs to cultivate different types of vegetables. After some studies the team determined that the optimal temperature for a good crop goes from 12 to 20 °C. To improve the greenhouse the team decided to develop a temperature control system with the following features:

- Turn on the heater if the temperature register is below 12 °C.
- Turn on the fan when the temperature goes above 20 °C.
- Turn off both devices if the temperature is optimal.

For this system we will use a red LED to simulate the heater and a small fan. Initially the temperature must be generated by a random number. After that replace the random number generator for a DHT22 sensor for temperature measurement.

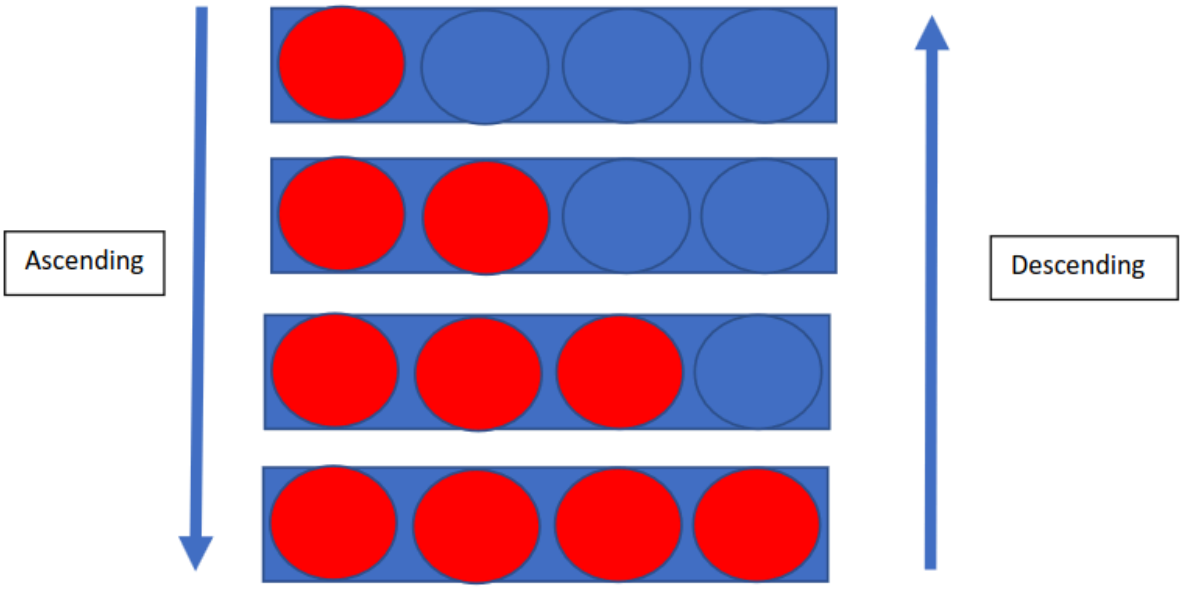
Exercise 4. Generate a program that will turn on 1 of 4 possible LEDs. To achieve this the system will have 2 buttons, the first one will select the LED to turn on and the second one will increase 1 second to the time the LED is turned on. When the first button is pressed the LED to activate will change to the next one and the time to turn on will reset to 1 second.

Implement functional programming.

Lab 3

Exercise 1. GPIO Tiva Exercise (User LEDS)

Using the 4 user LEDs and 2 user Switch make a simple LED sequence. With the first button make the LEDs to change their state in ascending order and with the second button change the LEDs in descending order.



Exercise 2. GPIO Tiva Exercise (User LED sequence)

Using the 4 user LEDs generate a loop sequence with 4 states minimum. Each state must be active for 2 seconds before it changes, use SysCtlDelay. How did you change the state duration? Is there a more efficient way to do this? (Investigate timers in TIVA)

Exercise 3. GPIO Tiva Exercise (Binary Counter)

Using the 4 user LEDs and 2 user Switch integrated on Tiva develop a binary counter with all the values showed in table 1 following the next features:

- When switch 1 is pressed, binary counter increases.
- When switch 2 is pressed, binary counter reduces.
- All 4 user LEDs represent the binary values.
- When counter reaches value 15 it shouldn't keep increasing.
- When counter goes to value 0 it shouldn't keep decreasing.
- Store the decimal value in an integer variable called "counter".

Lab 4

Raspberry Pi

- Using SSH create a new folder in your raspberry pi with the name "lab4", then make a python file inside the folder with the name "buzzer.py"

- Generate a python program in your computer that makes a buzzer sound when a button is pressed.
- Using SSH modify the previous program to make the buzzer sound when a button is pressed and stop when a second button is pressed.
- Create a led sequence that has a variable time interval between states. Make the program in a way that it can get ssh modifications without stopping the execution. Tip: Modify a txt file with ssh and read the file values in the main program from Raspberry Pi.

TIVA

Exercises:

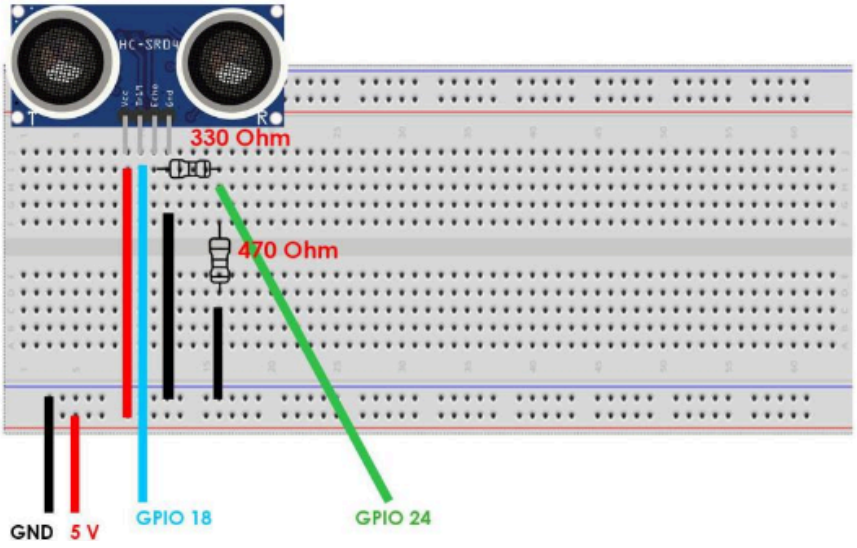
- Blink the user LED PN1 with **timer interruptions** in the Tiva. Initially blink the LED every second, then modify the code to blink the LED every 2 seconds and finally change the code to blink the LED every 5 seconds.
- Make a decimal sequence with the user leds that changes each 2 seconds.
- Modify the previous excercise to change the interval to 1 and 0,5 seconds when a user switch is pressed.
- Modify the binary counter exercise from lab 3 to use timers, each sequence should change without buttons and in a 1.5 second interval. Then add a new functionality, when a button is pressed change the time interval to 3 seconds.

Lab 5

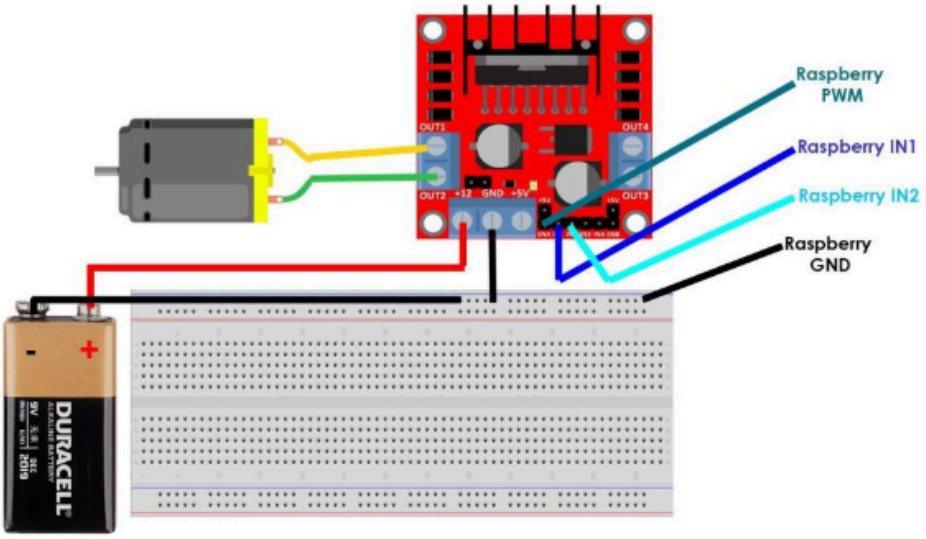
Exercises:

- **Connect a potentiometer to TIVA and make it vary the FS from a custom sequence using timer 0.**
- **Detect the distance of an object using the HC-SR04 ultrasonic sensor in Raspberry Pi. Then send the value of the distance to the Tiva with serial communication using UART. Finally follow the next instructions:**
    - **If the value of distance is above 10 cm, turn of all the user LEDs in the TIVA.**
    - **If the value of distance is between 10 – 8 cm, turn on the user LED PN1.**
    - **If the value of distance is between 8 – 6 cm, turn on the user LEDs PN1 and PN0.**
    - **If the value of distance is between 6 – 4 cm, turn on the user LEDs PN1, PN0 and PF4.**
    - **If the value of distance is below 4 cm, turn on all the user LEDs in the TIVA.**

You can use the following diagram to connect HC-SR04:



- **Using Pulse Width Modulation (PWM) change the speed of a DC motor following Figure 1. For this purpose, set the duty cycle to 75% and test the motor. Repeat this process with the following duty cycles: 25%, 45% and 50%. You can use the following schema for motor connection to raspberry Pi.**



- **Create a sequence using PWM to speed up a DC motor. The duty cycle of the motor must be initiated at 0% and every half of a second the value must increase by 1%. When the duty cycle is at his maximum, the value should return to 0.**
- **Using a potentiometer generate a PWM output and increase the intensity of an external LED (do not use any of the user LEDs) every half of a second. Finally, change the LED for a DC motor.**